



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Análisis y Diseño de Software I GT# :	Unidad 1	Tema: Fundamentos del análisis y Diseño de Software
	Semana #1	Tutor: Licdo. Edwin Aristondo Monzón González

Introducción:

Bienvenido al primer material de lectura de la Unidad #1: Fundamentos del Análisis y Diseño de Software. En este contenido se estudiará el concepto de Ingeniería de Software, sus niveles o capas, la crisis del software y sus principales causas y consecuencias.

Asimismo, se presentarán las fases generales del proceso de desarrollo de software y los principios fundamentales como la abstracción, modularidad, reutilización y calidad, los cuales permiten construir soluciones eficientes y bien estructuradas.

Este material te proporcionará una base sólida para comprender el Análisis y Diseño de Software y fortalecer tu formación académica y profesional mediante el desarrollo de las actividades propuestas.

[illegible]

[illegible]

Estas capas sustentan el planteamiento del IEEE al afirmar que la ingeniería de software es: la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software.

Sección 3:

Crisis del Software: Causas y consecuencias

La Crisis del software marcó un punto de inflexión en la historia de la informática. Este fenómeno, que surgió en la década de 1960, puso de manifiesto las limitaciones y desafíos inherentes al desarrollo de software a gran escala.

Qué provocó esta crisis? Fundamentalmente, la creciente complejidad de los sistemas informáticos superó la capacidad de los métodos de desarrollo existentes. Los proyectos de software a menudo excedían sus presupuestos, se entregaban tarde y no cumplían con las expectativas de los usuarios.

Esta situación generó una serie de problemas críticos:

1. Sobrecostos
2. Retrasos en la entrega
3. Baja calidad del software
4. Dificultad para mantener y actualizar sistemas
5. Falta de eficiencia en el uso de recursos
6. Insatisfacción de los usuarios finales
7. Aumento de la complejidad del código

La Crisis del software se refiere al período comprendido entre finales de los años 60 y mediados de los 80, caracterizado por los problemas generalizados en el desarrollo y mantenimiento de sistemas de software complejos.

La Crisis del Software (1960s - 1980s)

Periodo caracterizado por problemas generalizados en el desarrollo y mantenimiento de sistemas de software complejos

Problemas Principales

- Proyectos sobre presupuesto
- Plazos extendidos
- Software deficiente
- Código difícil de mantener
- Sistemas propensos a fallos
- Falta de profesionales calificados
- Comunicación deficiente

Soluciones Implementadas

- Ingeniería de Software formal
- Metodologías estructuradas
- Ciclos de vida del desarrollo
- Lenguajes de alto nivel
- Herramientas CASE
- Gestión de proyectos
- Control de calidad

Fases generales del Proceso de Desarrollo de Software

En el contexto de la ingeniería de software, un proceso no es una prescripción rígida de cómo elaborar software de cómputo. Por el contrario, es un enfoque adaptable que permite que el equipo de desarrollo busque y elija el conjunto apropiado de acciones y tareas para el trabajo. Se busca siempre entregar el software en forma oportuna y con calidad suficiente para satisfacer a quienes patrocinaron su creación y a aquellos que lo usarán.

La estructura del proceso establece el fundamento para el proceso completo de la ingeniería de software por medio de la identificación de un número pequeño de actividades estructurales que sean aplicables a todos los proyectos de software, sin importar su tamaño o complejidad.

Una estructura de proceso general para la ingeniería de software consta de cinco actividades:

1. **Comunicación.** Antes de que comience cualquier trabajo técnico, tiene importancia crítica comunicarse y colaborar con el cliente (y con otros participantes). Se busca entender los objetivos de los participantes respecto del proyecto, y reunir los requerimientos que ayuden a definir las características y funciones del software.
2. **Planeación.** Define el trabajo de ingeniería de software al describir las tareas técnicas por realizar, los riesgos probables, los recursos que se requieren, los productos del trabajo que se obtendrán y una programación de las actividades.
3. **Modelado:** Crea un "bosquejo" del objeto por hacer a fin de entender el panorama general. Un ingeniero de software crea modelos a fin de entender mejor los requerimientos del software y el diseño que los satisfará.
4. **Construcción:** Esta actividad combina la generación de código (ya sea manual o automatizada) y las pruebas que se requieren para descubrir errores en éste.

5. **Despliegue:** El software (como entidad completa o como un incremento parcialmente terminado) se entrega al consumidor que lo evalúa y que le da retroalimentación, misma que se basa en dicha evaluación.

Estas cinco actividades estructurales genéricas se usan durante el desarrollo de programas pequeños y sencillos, en la creación de aplicaciones web grandes y en la ingeniería de sistemas enormes y complejos basados en computadoras. Los detalles del proceso de software serán distintos en cada caso, pero las actividades estructurales son las mismas.

Principios de la Ingeniería de Software (Abstracción, modularidad, reutilización, calidad)

Estos principios ayudarán a los equipos de ingeniería de software a mantenerse concentrados, colaborar sin problemas e implementar productos más rápidamente.

1. Modularidad
2. Cohesión
3. Acoplamiento suelto
4. Escalabilidad
5. CI/CD
6. Abstracción
7. Encapsulación
8. Capacidad de prueba
9. SECO
10. SÓLIDO

Principio de ingeniería de software: abstracción

La abstracción es uno de los principios fundamentales de la ingeniería de software. Implica ocultar detalles complejos de implementación y garantizar que los usuarios interactúen con una interfaz simplificada.

Piense en la abstracción como un controlador de videojuego con solo unos pocos botones, pero con un mecanismo de control complejo en el fondo.

La abstracción es fundamental porque permite a los desarrolladores centrarse en los componentes esenciales del software e ignorar detalles innecesarios. Permite crear componentes de software más flexibles, modulares y reutilizables al separar la funcionalidad de alto nivel de los detalles de implementación de bajo nivel.

Principio de ingeniería de software: La modularidad

La modularidad es el principio de dividir una aplicación de software en pequeños módulos independientes. Estos módulos reciben nombres diferentes y se procesan de forma independiente antes de integrarse posteriormente para obtener un software completo y totalmente funcional.

El software de gran tamaño suele ser difícil de comprender y leer debido a la cantidad de rutas de control, variables de referencia y variables globales. La modularidad ayuda a reducir la complejidad general del código base al descomponerlo en partes más pequeñas y manejables.

Principio de ingeniería de Software: Calidad del software

La calidad del software es un principio fundamental de la ingeniería de software que busca garantizar que el sistema:

- Cumpla los requisitos del usuario
- Funcione correctamente
- Sea confiable y eficiente
- Sea fácil de mantener

Esto implica aplicar estándares, métricas, pruebas y buenas prácticas durante todo el proceso de desarrollo.

Principio de ingeniería de Software: Reutilización

Se habla de componentes de software como algoritmos, arquitecturas, diseños de prueba y desarrollo; estructuras de bases de datos, entre otros. La ingeniería del software basada en reutilización es una estrategia en la que el proceso de desarrollo se adapta a la utilización de software ya existente y ha surgido durante los últimos años como resultado de tendencias crecientes en la industria orientadas a reducir costo, riesgo y tiempo de desarrollo de aplicaciones; manteniendo la calidad final del producto y la satisfacción del cliente.

Bibliografía

Pressman, R. S. (2010). INGENIERIA DE SOFTWARE (7th ed.). McGraw-Hill

Interamericana de España S.L.

Sommerville, I. (2011). Ingeniería de software (9th ed.). Pearson Educación de México.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas (8.ª ed.). México: Pearson Educación.

